Week 10: Advanced SQL Query System

Scenario:

The stationery management system aims to perform advanced analysis on customer purchases, stationery product prices, and sales performance. The management team needs complex reports to identify high-value customers, costly stationery products, and business trends. Students will implement Subqueries and Nested Queries to extract deeper insights from the Stationery Management Database.

1. Implement Subqueries and Nested Queries Concept:

Single-row Subquery

```
SQL> SELECT product_name, price FROM product WHERE price > (SELECT AVG(price) FROM product);

PRODUCT_NAME               PRICE
-------------------------- ------
Pencil Box                     60
Spiral Notebook                60
Notebook                       55
```

```
SQL> SELECT product_name, price FROM product WHERE price = (SELECT MAX(price) FROM product);

PRODUCT_NAME               PRICE
-------------------------- ------
Pencil Box                     60
Spiral Notebook                60
```

Multi-row Subquery

```
SQL> SELECT product_name, price FROM product WHERE category_id IN (SELECT category_id FROM category);

PRODUCT_NAME               PRICE
-------------------------- ------
Ball Pen                       20
Gel Pen                        15
Pencil Box                     60
Spiral Notebook                60
Notebook                       55
Pencil                          8

6 rows selected.
```

Nested Subquery

```
SQL> SELECT product_name, price FROM product WHERE price > (SELECT AVG(price) FROM product WHERE category_id IN (SELECT category_id FROM category));

PRODUCT_NAME               PRICE
-------------------------- ------
Pencil Box                     60
Spiral Notebook                60
Notebook                       55
```

2. Find Products Above Average Price Task:

- Find products priced above the average price

```
SQL> SELECT product_id, product_name, price FROM product WHERE price > (SELECT AVG(price) FROM product);

PRODUCT PRODUCT_NAME               PRICE
------- -------------------------- ------
P004    Pencil Box                     60
P005    Spiral Notebook                60
P016    Notebook                       55
```

- Display product name, category, and price.

```
SQL> SELECT p.product_name, c.category_name, p.price FROM product p JOIN category c ON p.category_id = c.category_id WHERE p.price > (SELECT AVG(pri
ce) FROM product);

PRODUCT_NAME               CATEGORY_NAME                                    PRICE
-------------------------- ---------------------------------------------------------------------------------------------------- ------
Pencil Box                 Art Supplies                                         60
Spiral Notebook            Files and Folders                                    60
Notebook                   Notebooks                                            55
```

- Identify expensive products in each category.

```
SQL> SELECT p.product_name, p.category_id, p.price FROM product p WHERE p.price = (SELECT MAX(p2.price) FROM product p2 WHERE p2.category_id = p.cat
egory_id);

PRODUCT_NAME             CATEGOR  PRICE
------------------------ ------- ------
Ball Pen                 101         20
Pencil Box               104         60
Spiral Notebook          105         60
Notebook                 102         55
```

- Find products with price higher than a selected category average.

```
SQL> SELECT p.product_name, c.category_name, p.price FROM product p JOIN category c ON p.category_id = c.category_id WHERE p.price > (SELECT AVG(p2.
price) FROM product p2 WHERE p2.category_id = p.category_id);

PRODUCT_NAME             CATEGORY_NAME                                                                                                        PRICE
------------------------ ---------------------------------------------------------------------------------------------------------------- ------
Ball Pen                 Writing Instruments                                                                                                    20
Notebook                 Notebooks                                                                                                              55
```

3. Electronics Identify Customers with Maximum Purchases Task

- Customer who spent the highest amount.

```
SQL> SELECT c.customer_id, c.customer_name, SUM(o.total_amount) AS total_spent FROM customer c JOIN orders o ON c.customer_id = o.customer_id GROUP
BY c.customer_id, c.customer_name HAVING SUM(o.total_amount) = (SELECT MAX(total_sales) FROM (SELECT SUM(total_amount) AS total_sales FROM orders GR
OUP BY customer_id));

CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_SPENT
----------- -------------------------------------------------- -----------
        105 Ram                                                       5071
```

- Customers with maximum number of orders.

```
SQL> SELECT c.customer_id, c.customer_name, COUNT(o.order_id) AS total_orders FROM customer c JOIN orders o ON c.customer_id = o.customer_id GROUP B
Y c.customer_id, c.customer_name HAVING COUNT(o.order_id) = (SELECT MAX(order_count) FROM (SELECT COUNT(*) AS order_count FROM orders GROUP BY custo
mer_id));

CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_ORDERS
----------- -------------------------------------------------- ------------
        103 Rahul Raj                                                     1
        102 Priya Sharma                                                  1
        104 Divya Devi                                                    1
        101 Arun Kumar Raj                                                1
        105 Ram                                                           1
```

- Customers whose spending is above average.

```
SQL> SELECT c.customer_id, c.customer_name, SUM(o.total_amount) AS total_spent FROM customer c JOIN orders o ON c.customer_id = o.customer_id GROUP
BY c.customer_id, c.customer_name HAVING SUM(o.total_amount) > (SELECT AVG(customer_total) FROM (SELECT SUM(total_amount) AS customer_total FROM ord
ers GROUP BY customer_id));

CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_SPENT
----------- -------------------------------------------------- -----------
        103 Rahul Raj                                                4800
        105 Ram                                                      5071
```

- Top 5 valuable customers.

```
SQL> SELECT * FROM (SELECT c.customer_id, c.customer_name, SUM(o.total_amount) AS total_spent FROM customer c JOIN orders o ON c.customer_id = o.cus
tomer_id GROUP BY c.customer_id, c.customer_name ORDER BY total_spent DESC) WHERE ROWNUM <= 5;

CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_SPENT
----------- -------------------------------------------------- -----------
        105 Ram                                                      5071
        103 Rahul Raj                                                4800
        101 Arun Kumar Raj                                           2500
        104 Divya Devi                                                400
        102 Priya Sharma                                              307
```

4.Generate Complex Business Queries

Query 1: Best-Selling Product

Find products with maximum sales quantity.

Tasks: ● Display product name.

- Total quantity sold.
- Total revenue generated.

```
SQL> SELECT p.product_name, SUM(od.quantity) AS total_quantity_sold, SUM(od.quantity * od.price) AS total_revenue FROM product p JOIN order_details
od ON p.product_id = od.product_id GROUP BY p.product_id, p.product_name HAVING SUM(od.quantity) = (SELECT MAX(total_quantity) FROM (SELECT SUM(quan
tity) AS total_quantity FROM order_details GROUP BY product_id));

PRODUCT_NAME              TOTAL_QUANTITY_SOLD TOTAL_REVENUE
------------------------- ------------------- -------------
Pencil Box                                 20         80000
```

Query 2: High-Value Customers

Find customers whose total purchase amount is greater than average customer spending.

```
SQL> SELECT c.customer_id, c.customer_name, SUM(o.total_amount) AS total_spending FROM customer c JOIN orders o ON c.customer_id = o.customer_id GRO
UP BY c.customer_id, c.customer_name HAVING SUM(o.total_amount) > (SELECT AVG(customer_spending) FROM (SELECT SUM(total_amount) AS customer_spending
 FROM orders GROUP BY customer_id));

CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_SPENDING
----------- -------------------------------------------------- --------------
        103 Rahul Raj                                                    4800
        105 Ram                                                          5071
```

Query 3: Category Performance Analysis

Find:

- Highest revenue-generating category.

```
SQL> SELECT * FROM (SELECT c.category_name, NVL(SUM(od.quantity * od.price), 0) AS total_revenue FROM category c LEFT JOIN product p ON c.category_i
d = p.category_id LEFT JOIN order_details od ON p.product_id = od.product_id GROUP BY c.category_id, c.category_name ORDER BY total_revenue DESC) WH
ERE ROWNUM = 1;

CATEGORY_NAME                                                                                                    TOTAL_REVENUE
---------------------------------------------------------------------------------------------------------------- -------------
Art Supplies                                                                                                        80000
```

- Number of products sold in each category.

```
SQL> SELECT c.category_name, NVL(SUM(od.quantity), 0) AS products_sold FROM category c LEFT JOIN product p ON c.category_id = p.category_id LEFT JOI
N order_details od ON p.product_id = od.product_id GROUP BY c.category_id, c.category_name;

CATEGORY_NAME                                                                                                    PRODUCTS_SOLD
---------------------------------------------------------------------------------------------------------------- -------------
Art Supplies                                                                                                            20
Files and Folders                                                                                                       14
Paper Products                                                                                                           0
Office Supplies                                                                                                          0
Writing Instruments                                                                                                     20
Adhesives                                                                                                                0
Desk Accessories                                                                                                         0
School Supplies                                                                                                          0
Notebooks                                                                                                               10

9 rows selected.
```

- Average product price by category.

```
SQL> SELECT c.category_name, AVG(p.price) AS average_product_price FROM category c JOIN product p ON c.category_id = p.category_id GROUP BY c.catego
ry_id, c.category_name;

CATEGORY_NAME                                                                                                    AVERAGE_PRODUCT_PRICE
---------------------------------------------------------------------------------------------------------------- ---------------------
Art Supplies                                                                                                                    60
Files and Folders                                                                                                               60
Writing Instruments                                                                                                             14
Notebooks                                                                                                                       35
```

Query 4: Customer Purchase History

Display:

- Customer name
- Number of orders
- Total purchase amount

- Most purchased product

```
SQL> WITH order_totals AS (SELECT customer_id, COUNT(order_id) AS number_of_orders, SUM(total_amount) AS total_purchase_amount FROM orders GROUP BY
customer_id),
  2  product_totals AS (SELECT o.customer_id, p.product_name, SUM(od.quantity) AS total_quantity, ROW_NUMBER() OVER (PARTITION BY o.customer_id ORDE
R BY SUM(od.quantity) DESC, p.product_name) AS rn FROM orders o JOIN order_details od ON o.order_id = od.order_id JOIN product p ON od.product_id =
p.product_id GROUP BY o.customer_id, p.product_id, p.product_name)
  3  SELECT c.customer_name, NVL(ot.number_of_orders, 0) AS number_of_orders, NVL(ot.total_purchase_amount, 0) AS total_purchase_amount, pt.product_
name AS most_purchased_product FROM customer c LEFT JOIN order_totals ot ON c.customer_id = ot.customer_id LEFT JOIN product_totals pt ON c.customer
_id = pt.customer_id AND pt.rn = 1;

CUSTOMER_NAME          | NUMBER_OF_ORDERS | TOTAL_PURCHASE_AMOUNT | MOST_PURCHASED_PRODUCT
---------------------- | ---------------- | --------------------- | ----------------------
Arun Kumar Raj         |                1 |                  2500 | Ball Pen
Priya Sharma           |                1 |                   307 | Gel Pen
Rahul Raj              |                1 |                  4800 | Pencil
Divya Devi             |                1 |                   400 | Pencil Box
Ram                    |                1 |                  5071 | Spiral Notebook
```

5. Prepare Advanced SQL Report

Report 1: Premium Product Report

Display:

- Product Name
- Category
- Price
- Stock Availability Condition:

  Products above average price.

```
SQL> SELECT p.product_name, c.category_name, p.price, CASE WHEN p.stock_quantity > 0 THEN 'Available' ELSE 'Out of Stock' END AS stock_availability
FROM product p JOIN category c ON p.category_id = c.category_id WHERE p.price > (SELECT AVG(price) FROM product);

PRODUCT_NAME
--------------------------------------------------------------------------------------------------------------------------------------------------------
--
CATEGORY_NAME                                                                                                                    |      PRICE | STOCK_AVAILA
-------------------------------------------------------------------------------------------------------------------------------- | ---------- | ------------
Pencil Box
Art Supplies                                                                                                                     |         60 | Available

Spiral Notebook
Files and Folders                                                                                                                |         60 | Available

Notebook
Notebooks                                                                                                                        |         55 | Available
```

Report 2: Customer Value Report

Display:

- Customer Name
- Total Orders
- Total Spending
- Customer Category

```
SQL> SELECT c.customer_name, COUNT(o.order_id) AS total_orders, NVL(SUM(o.total_amount), 0) AS total_spending, CASE WHEN NVL(SUM(o.total_amount), 0)
 >= 50000 THEN 'Premium' ELSE 'Regular' END AS customer_category FROM customer c LEFT JOIN orders o ON c.customer_id = o.customer_id GROUP BY c.cust
omer_id, c.customer_name;

CUSTOMER_NAME          | TOTAL_ORDERS | TOTAL_SPENDING | CUSTOME
---------------------- | ------------ | -------------- | -------
Rahul Raj              |            1 |           4800 | Regular
Priya Sharma           |            1 |            307 | Regular
Divya Devi             |            1 |            400 | Regular
Arun Kumar Raj         |            1 |           2500 | Regular
Ram                    |            1 |           5071 | Regular
```

Report 3: Sales Performance Report

Display:

- Product-wise sales

```
SQL> SELECT p.product_name, NVL(SUM(od.quantity), 0) AS quantity_sold, NVL(SUM(od.quantity * od.price), 0) AS total_revenue FROM product p LEFT JOIN
 order_details od ON p.product_id = od.product_id GROUP BY p.product_id, p.product_name;

PRODUCT_NAME
   | QUANTITY_SOLD | TOTAL_REVENUE
-------------------------------------------------------------------------------------------------------------------------------------------------
-- | ------------- | -------------
Notebook
   |             0 |             0
Ball Pen
   |             5 |          5000
Spiral Notebook
   |            14 |         21000
Pencil
   |            15 |         45000
Pencil Box
   |            20 |         80000
Gel Pen
   |            10 |         20000

6 rows selected.
```

- Category-wise revenue

```
SQL> SELECT c.category_name, NVL(SUM(od.quantity * od.price), 0) AS total_revenue FROM category c LEFT JOIN product p ON c.category_id = p.category_
id LEFT JOIN order_details od ON p.product_id = od.product_id GROUP BY c.category_id, c.category_name;

CATEGORY_NAME                                                                                       | TOTAL_REVENUE
--------------------------------------------------------------------------------------------------- | -------------
Art Supplies                                                                                        |         80000
Files and Folders                                                                                   |         21000
Paper Products                                                                                      |             0
Office Supplies                                                                                     |             0
Writing Instruments                                                                                 |         50000
Adhesives                                                                                           |             0
Desk Accessories                                                                                    |             0
School Supplies                                                                                     |             0
Notebooks                                                                                           |         20000

9 rows selected.
```

- Top-selling products

```
SQL> SELECT * FROM (SELECT p.product_name, SUM(od.quantity) AS quantity_sold FROM product p JOIN order_details od ON p.product_id = od.product_id GR
OUP BY p.product_id, p.product_name ORDER BY quantity_sold DESC) WHERE ROWNUM <= 5;

PRODUCT_NAME
   | QUANTITY_SOLD
-------------------------------------------------------------------------------------------------------------------------------------------------
-- | -------------
Pencil Box
   |            20
Pencil
   |            15
Spiral Notebook
   |            14
Gel Pen
   |            10
Ball Pen
   |             5
```

Report 4: Business Decision Report

Identify:

- High-performing products

```
SQL> SELECT p.product_name, SUM(od.quantity) AS quantity_sold, SUM(od.quantity * od.price) AS total_revenue FROM product p JOIN order_details od ON
p.product_id = od.product_id GROUP BY p.product_id, p.product_name HAVING SUM(od.quantity) >= (SELECT AVG(product_quantity) FROM (SELECT SUM(quantit
y) AS product_quantity FROM order_details GROUP BY product_id)) ORDER BY total_revenue DESC;

PRODUCT_NAME
   | QUANTITY_SOLD | TOTAL_REVENUE
-------------------------------------------------------------------------------------------------------------------------------------------------
-- | ------------- | -------------
Pencil Box
   |            20 |         80000
Pencil
   |            15 |         45000
Spiral Notebook
   |            14 |         21000
```

- Low-performing products

```
SQL> SELECT p.product_name, NVL(SUM(od.quantity), 0) AS quantity_sold, NVL(SUM(od.quantity * od.price), 0) AS total_revenue FROM product p LEFT JOIN
 order_details od ON p.product_id = od.product_id GROUP BY p.product_id, p.product_name HAVING NVL(SUM(od.quantity), 0) <= (SELECT AVG(product_quant
ity) FROM (SELECT SUM(quantity) AS product_quantity FROM order_details GROUP BY product_id));

PRODUCT_NAME
   | QUANTITY_SOLD | TOTAL_REVENUE
-------------------------------------------------------------------------------------------------------------------------------------------------
-- | ------------- | -------------
Notebook
   |         0 |             0
Ball Pen
   |         5 |          5000
Gel Pen
   |        10 |         20000
```

- High-value customers

```
SQL> SELECT c.customer_name, SUM(o.total_amount) AS total_spending FROM customer c JOIN orders o ON c.customer_id = o.customer_id GROUP BY c.custome
r_id, c.customer_name HAVING SUM(o.total_amount) > (SELECT AVG(customer_spending) FROM (SELECT SUM(total_amount) AS customer_spending FROM orders GR
OUP BY customer_id)) ORDER BY total_spending DESC;

CUSTOMER_NAME          | TOTAL_SPENDING
---------------------- | --------------
Ram                    |           5071
Rahul Raj              |           4800
```

- Revenue trends

```
SQL> SELECT TO_CHAR(order_date, 'YYYY-MM') AS sales_month, COUNT(order_id) AS total_orders, SUM(total_amount) AS monthly_revenue FROM orders GROUP B
Y TO_CHAR(order_date, 'YYYY-MM') ORDER BY sales_month;

SALES_M | TOTAL_ORDERS | MONTHLY_REVENUE
------- | ------------ | ---------------
2025-01 |            1 |            2500
2025-02 |            1 |             307
2025-03 |            1 |            4800
2025-04 |            1 |             400
2025-05 |            1 |            5071
```